

软件设计规格说明书

项目名称：校园搭子任务互助平台

小组名称及成员：第 5 组

课程名称：软件工程

提交日期：2026-05-22

版本号：v1.0

摘要

本文档是校园搭子任务互助平台（Task Buddy）的软件设计规格说明书。系统采用微信小程序 + Node.js/Express + MySQL 的三层前后端分离架构，面向校园学生提供任务发布、搭子配对、进度监督、证据提交、互相评价与积分激励功能。本文档涵盖系统架构设计（模块划分、接口定义、技术选型）、界面设计（用户分析、界面规范、原型说明、交互流程）及详细设计（核心用例的完整流程、类设计、数据库 E-R 图与逻辑结构、关键算法），为后续编码实现提供直接依据。

1 引言

1.1 编写目的

本文档旨在将已评审通过的需求规格转化为可指导编码的设计方案，明确系统的架构选型、模块划分、接口契约、界面规范与核心算法，供开发、测试、评审各方使用。

1.2 项目背景

校园学生普遍存在任务拖延、缺乏监督的问题。Task Buddy 以微信小程序为载体，通过“搭子监督 + 积分激励 + 排名机制”构建正向反馈闭环，帮助学生提升任务完成率。前序阶段已完成需求获取与需求规格说明书（v1.0，第 5 组，2026-05-13）的评审。

1.3 参考资料

- 《校园搭子任务互助平台——需求规格说明书》v1.0，第 5 组，2026-05-13
- 微信小程序官方开发文档（developers.weixin.qq.com）
- Express.js 官方文档（expressjs.com）
- MySQL 8.0 参考手册
- 《软件设计》课程讲义

1.4 术语定义

术语	定义
搭子	加入他人任务贴的监督伙伴
任务贴 / 帖子	发布者创建的任务信息载体，具有完整生命周期状态
积分结算	任务终态触发的积分增减操作，要求原子性
推荐分	系统为每篇帖子实时计算的综合排序分值
openid	微信平台为每个用户分配的全局唯一标识符
待评价	任务结束后等待双方互评的中间状态

1.5 小组分工

成员	架构设计	界面设计	详细设计
成员 A	主负责	参与	参与
成员 B	参与	主负责	参与
成员 C	参与	参与	主负责
成员 D	参与	参与	参与

2 系统架构设计

2.1 需求回顾

核心功能需求（P0）：微信授权登录注册（R01/R02）、任务贴发布与浏览（R03/R04）、搭子加入/退出（R07/R08）、证据上传（R10）、积分结算（R14/R15）、发布者放弃（R13）。

核心非功能需求：写操作响应 ≤ 1.5 秒、读操作 ≤ 1 秒、支持 200 并发、小程序主包 ≤ 2 MB、积分操作原子性、帖子内容 100% 接入微信内容安全接口。

2.2 架构选型分析

2.2.1 备选架构模式对比

对比维度	方案 A：传统 MVC 单体	方案 B：前后端分离（本项目）
前端渲染	服务端模板渲染	客户端渲染（微信小程序）
接口形式	页面路由 + 表单提交	RESTful JSON API
部署方式	前后端同进程	前端在微信平台，后端独立部署
扩展性	前后端耦合，扩展困难	各层独立演进
适配微信小程序	不适合（小程序无浏览器渲染）	天然契合
学习成本	低	中（需理解 REST 约定）

方案 C：Serverless（云函数）：微信云开发可零运维，但免费配额有限，数据库能力弱，不适合复杂 SQL 查询（推荐算法、排名计算）。

2.2.2 最终架构选择及理由

- 选择方案 B：前后端分离架构。理由：
- 微信小程序本身就是独立客户端，强制前后分离；
 - RESTful API 便于分工开发和独立测试；
 - Node.js + Express 开发效率高，异步 I/O 适合高并发读操作；
 - MySQL 满足复杂查询（多表 JOIN、事务）和数据一致性要求。

2.3 系统总体结构



2.3.1 子系统 / 模块划分

前端模块:

模块	路径	职责
登录模块	pages/login	微信授权、首次绑定昵称头像
首页模块	pages/home	帖子列表展示、搜索筛选、推荐排序
帖子详情模块	pages/post-detail	任务全生命周期操作 (加入/退出/证据/评价)
发布模块	pages/publish	创建和编辑帖子
排行榜模块	pages/ranking	积分排名展示
个人中心模块	pages/profile	个人信息、历史帖子、积分记录
API 服务层	services/api.js	封装所有 HTTP 请求, 统一错误处理

后端模块:

模块	位置	职责
----	----	----

认证模块	server.js: /api/auth/*	微信 code 换 openid、用户注册登录
帖子管理模块	server.js: /api/posts/*	帖子 CRUD、状态流转
搭子互动模块	server.js: /api/posts/:id/join,quit	加入/退出搭子关系
证据评价模块	server.js: /api/posts/:id/evidence,evaluate	证据提交、互评处理
积分结算模块	server.js: settlePost, recalcCompletionRate	原子性积分操作、完成率重算
推荐算法模块	server.js: calcRecommendedScore	帖子推荐分实时计算
用户模块	server.js: /api/users/*	个人主页、排名查询
数据访问层	src/db.js	连接池管理、建表、事务封装

2.3.2 子系统职责说明

- **认证模块：**调用微信 jscode2session 接口换取 openid；开发环境自动降级（以昵称为 key 构造 dev_* openid）。新用户返回 isNewUser: true，由前端引导绑定。
- **帖子管理模块：**维护帖子状态机（招募中 → 进行中 → 待评价 → 已完成 / 已放弃），每次读写帖子时调用 syncPostStatus 检查时间触发条件。
- **积分结算模块：**使用数据库事务保证“状态更新 + 积分变更”的原子性，防止网络闪断导致数据不一致。
- **推荐算法模块：**综合发布者信用（完成率×0.6 + 积分÷10×0.4）、帖子质量（奖励分 + 惩罚分 + 热度加成）和用户偏好（历史参与类别）三维加权，实时为每篇帖子计算推荐分。

2.4 接口设计

2.4.1 子系统间接口

前端通过 services/api.js 统一调用后端，所有接口遵循：

- 基础路径：http://localhost:3000/api
- 数据格式：JSON（UTF-8）
- 认证方式：请求体或 Query 传递 userId（token 格式：token-{userId}，存于 wx.setStorageSync）
- 错误响应：{ message: "错误描述" }，HTTP 状态码 4xx/5xx

完整接口清单：

方法	路径	功能
POST	/api/auth/login	微信 code 换取用户信息
POST	/api/auth/bind	首次登录绑定，创建用户

GET	/api/posts	获取帖子列表（分页、筛选、推荐排序）
POST	/api/posts	发布新帖子
GET	/api/posts/:id	获取帖子详情（含证据、搭子、互评）
PUT	/api/posts/:id	编辑帖子
DELETE	/api/posts/:id	删除帖子
POST	/api/posts/:id/join	搭子加入任务
POST	/api/posts/:id/quit	搭子退出任务
POST	/api/posts/:id/start	发布者手动开始任务
POST	/api/posts/:id/complete	发布者标记完成 → 待评价
POST	/api/posts/:id/request-complete	搭子申请完成
POST	/api/posts/:id/evidence	提交任务证据
POST	/api/posts/:id/evaluate	提交互评
POST	/api/posts/:id/abandon	发布者放弃任务
GET	/api/ranking	获取积分排名
GET	/api/users/:id/profile	获取用户信息与帖子列表
PUT	/api/users/:id/profile	更新用户昵称头像
GET	/api/users/:id/evaluations-received	获取收到的评价列表
GET	/api/health	健康检查

2.4.2 外部接口

接口	提供方	用途
jscode2session	微信开放平台	code 换 openid, 超时 3 秒后开发降级
微信内容安全 API	微信开放平台	帖子文本违规检测（P0 需求）
wx.chooseMedia	微信小程序 SDK	选择本地图片作为证据
wx.uploadFile	微信小程序 SDK	上传图片至后端

2.5 关键技术与设计决策

2.5.1 技术栈选择

层次	技术	版本	选择理由
前端	微信小程序 (WXML/WXSS/JS)	基础库 3.x	目标用户使用微信，零下载成本
后端框架	Node.js + Express	Node 18+, Express 4.x	异步 I/O 性能好，生态成熟
数据库	MySQL 8.0	8.0+	支持事务、外键、复杂 JOIN，满足一致

			性要求
ORM/驱动	mysql2/promise	3. x	原生 Promise，支持连接池和预处理语句
HTTP 客户端	axios	1. x	调用微信开放平台接口

2.5.2 重要设计决策记录

决策 1：帖子状态由服务端主动同步（`syncPostStatus`）而非定时任务

- 问题：帖子超时需自动流转状态（进行中 → 待评价），但小程序不能长驻后台运行定时器。
- 决策：每次读写帖子时在服务端调用 `syncPostStatus`，检查 `endTime` 和 `evaluationDeadline`，按需流转。
- 权衡：实现简单，无需额外进程；但状态变更不实时，需用户触发请求才生效。

决策 2：积分结算使用数据库悲观锁事务（`withTransaction`）

- 问题：积分是核心激励数据，要求“任务完成 → 积分增加”原子执行。
- 决策：通过 `UPDATE posts SET status='已完成' WHERE id=? AND status='待评价'` 的原子 `UPDATE` 实现乐观锁，`affectedRows === 0` 时跳过，防止重复结算。
- 权衡：避免分布式锁复杂度，适合单机部署场景。

决策 3：推荐分在 API 请求时实时计算（内存排序），不持久化

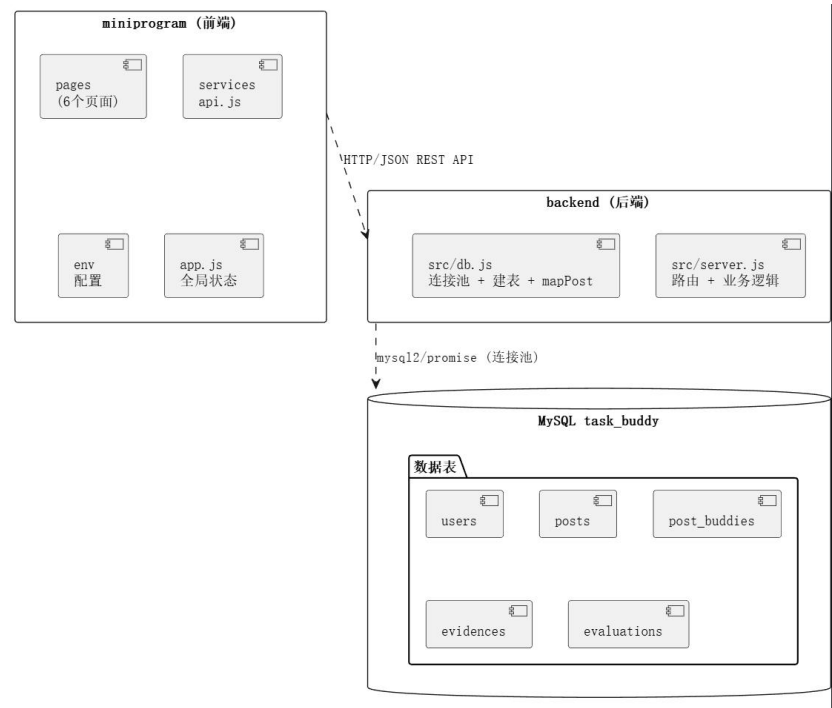
- 问题：推荐分依赖发布者当前积分/完成率和调用者的历史偏好，是动态值。
- 决策：取全量候选帖子到内存，实时计算后排序分页，不写回数据库 `recommendedScore` 字段。
- 权衡：帖子量小（校园场景）时性能可接受；量大时需改为异步预计算。

2.5.3 技术风险分析

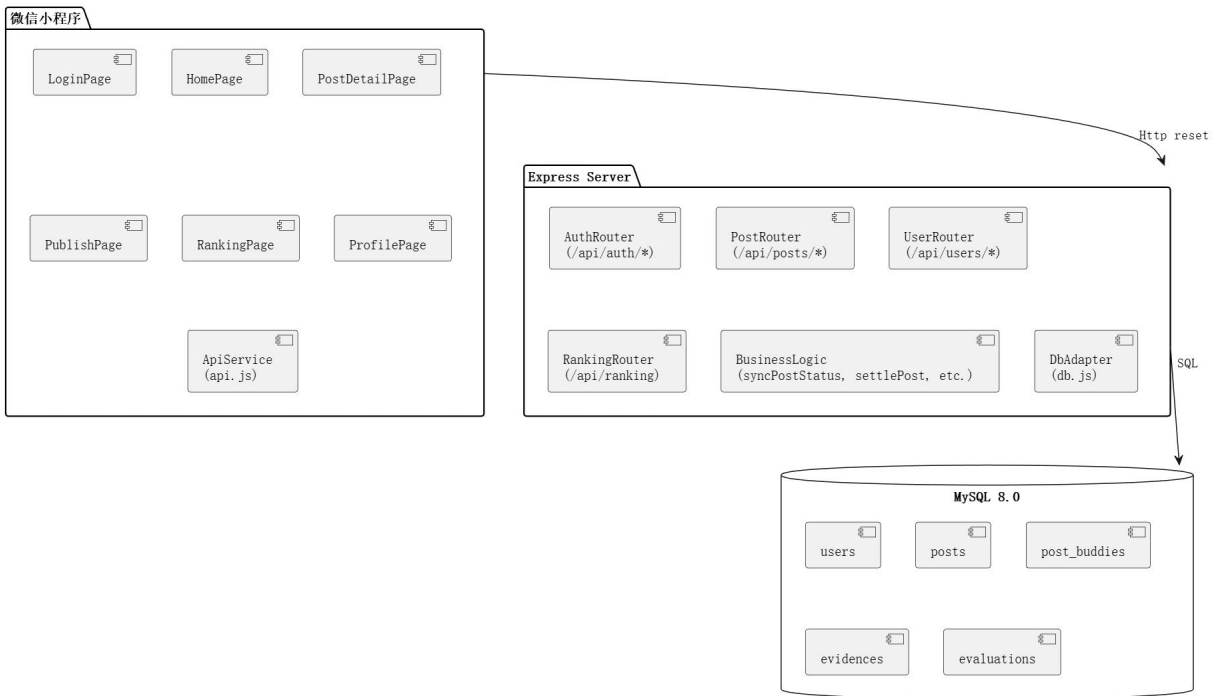
风险	影响	应对措施
微信 <code>jscode2session</code> 超时或失败	用户无法登录	设置 3 秒超时，开发环境自动降级为 <code>dev_* openid</code>
积分重复结算	数据不一致	乐观锁：原子 <code>UPDATE</code> + <code>affectedRows</code> 检查
内存推荐排序性能	帖子量大时响应慢	校园场景帖子量有限；长期方案：异步预计算 + 缓存
MySQL 连接耗尽	服务不可用	连接池 <code>connectionLimit</code> : 10，队列等待模式

2.6 系统架构 UML 模型

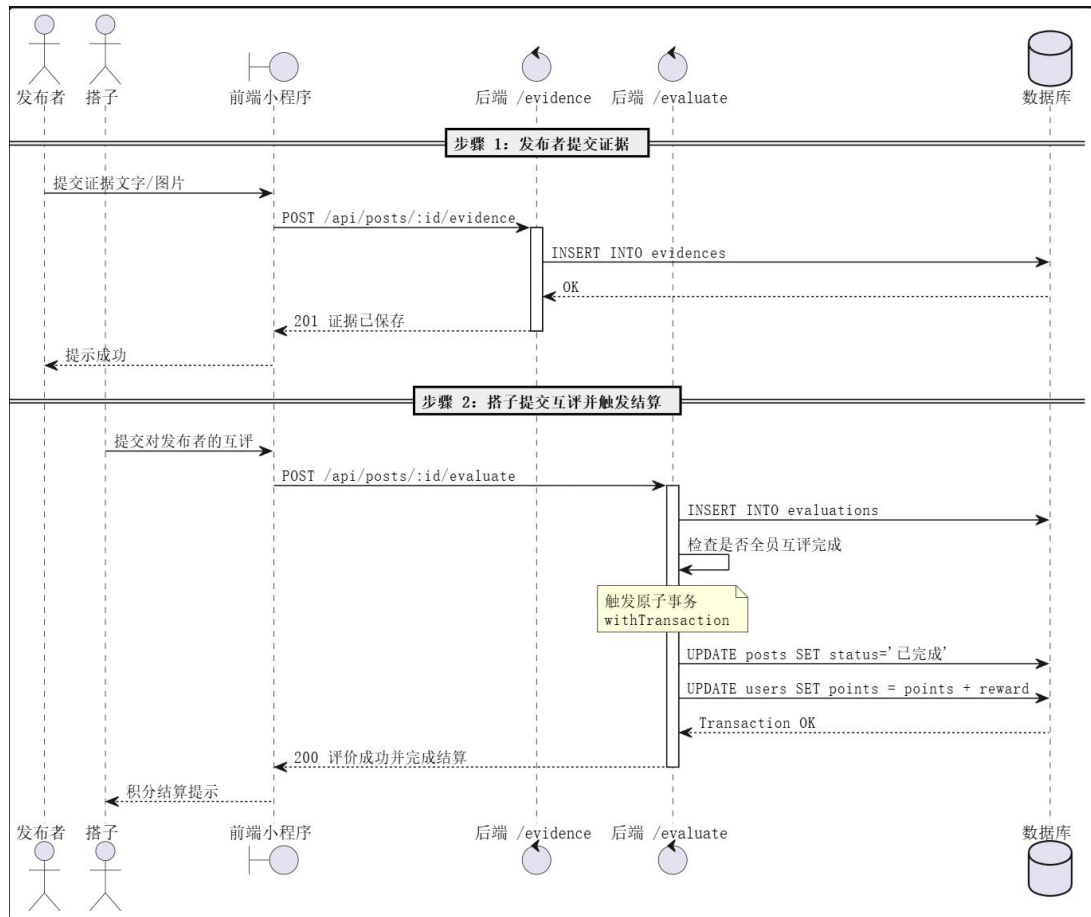
2.6.1 系统包图



2.6.2 组件图



2.6.3 核心业务高层顺序图（任务完成全流程）



3 界面设计

3.1 用户分析

3.1.1 目标用户特征

属性	描述
年龄	18 - 22 岁在校大学生
设备	iOS 12+ / Android 8.0+, 微信 7.0+
使用场景	自习组队、晨跑打卡、考研复习、求职备战
核心痛点	拖延、缺乏监督、找不到学习伙伴
操作习惯	熟悉微信生态，习惯单手操作，期望即时反馈

3.1.2 用户使用习惯与需求

- 单次使用时长短（5 分钟内完成发帖/加入/打卡），要求交互路径极简。
- 偏好视觉化进度反馈（状态标签、积分变化提示）。

- 对冗余操作步骤容忍度低，核心流程不超过 3 次点击（需求 4.2）。

3.2 界面风格与规范

3.2.1 整体风格定位

校园清新风：浅色背景 + 主题色蓝（#1a6cf5）+ 辅助色橙（积分、徽章）。整体布局简洁，信息层级清晰，避免过度装饰。

3.2.2 设计规范

规范项	规范值
主题色	#1a6cf5（蓝）
辅助色	#f5a623（橙，积分相关）
危险色	#e74c3c（红，放弃/扣分）
成功色	#27ae60（绿，完成）
背景色	#f5f7fa（浅灰）
主字体	系统默认字体，正文 28rpx，标题 32rpx，辅助 24rpx
圆角	卡片 16rpx，按钮 8rpx
间距	页面边距 24rpx，卡片内间距 20rpx
图标	微信 WeUI 图标库 + 项目自定义图标
按钮规范	主按钮高度 88rpx，宽度 100%；次按钮高度 72rpx

3.3 界面原型设计

3.3.1 主界面设计

登录页（/pages/login/login）

- 顶部 Logo + 标语区
- 中部微信头像占位圆 + 昵称输入框（首次绑定）
- 底部「微信一键登录」主按钮（蓝色，全宽）
- 错误提示区（红色文字，输入验证失败时显示）

首页 / 任务广场（/pages/home/home）

- 顶部：搜索框 + 筛选按钮（漏斗图标）
- 筛选面板（下拉展开）：类别标签栏（学习/运动/考研/求职/自律）+ 时间范围选择器
- 用户积分状态栏：当前积分、完成率、排名（横向三格卡片）
- 推荐提示标签：「基于任务类型、完成率、互评质量的推荐模型」
- 帖子卡片列表（无限下拉加载）

- 卡片内容：标题、类别标签、发布者头像+昵称、奖励积分（橙色醒目）、截止时间、状态标签
 - 底部 TabBar：任务广场 / 排行榜 / 个人中心
- 帖子详情页 (/pages/post-detail/post-detail)**
- 布局分区（从上到下）：
5. 帖子基本信息区：标题、内容、类别、奖励/罚分、时间
 6. 发布者信息区：头像、昵称、完成率
 7. 搭子信息区：已加入搭子列表
 8. 操作按钮区（按角色和状态动态显示）：
 - 普通用户 + 招募中：「加入搭子」
 - 发布者 + 招募中：「开始任务」「放弃任务」
 - 发布者 + 进行中：「标记完成」「放弃任务」
 - 搭子 + 进行中：「申请完成」「退出任务」
 - 参与者 + 待评价：「提交证据」「提交互评」
 9. 证据展示区（待评价/已完成状态）
 10. 互评区（已完成状态）：发送的评价 + 收到的评价

3.3.2 功能模块界面设计

发布页 (/pages/publish/publish)

- 表单字段（从上到下）：
 - 标题输入框（必填，最多 255 字符）
 - 内容输入框（多行文本，必填）
 - 类别选择器（Picker：学习/运动/考研/求职/自律）
 - 奖励积分 / 惩罚积分（数字输入，最小 0）
 - 开始时间 / 结束时间（日期时间选择器）
 - 最大搭子人数（数字输入，默认 1）
 - 是否开启搭子聊天（Switch）
 - 是否开启互评（Switch）
 - 任务证据要求（多行文本）
- 底部「发布」主按钮

排行榜页 (/pages/ranking/ranking)

- 顶部 Top3 大卡片（头像 + 昵称 + 积分 + 名次）
- 下方完整排名列表（序号 + 头像 + 昵称 + 积分 + 完成率）

个人中心页 (/pages/profile/profile)

- 顶部：头像（可更换）+ 昵称（可编辑）+ 积分 + 排名

- AI 评价卡片（浅蓝背景，斜体文字）
- 任务完成率进度条
- 帖子列表（Tab 切换：发布的 / 参与的）

3.3.3 关键操作界面设计

证据提交弹层：底部上滑弹层，文字输入框（多行），「提交」按钮。

互评人员选择弹层：底部弹层，展示所有参与者（头像 + 昵称），点击进入评分表单。

评分表单：显示被评价者信息，星级评分（1-5 星滑动选择），评语输入框，「提交评价」按钮。

3.4 界面交互设计

3.4.1 交互流程说明

核心流程 1：新用户注册并发布任务（≤ 3 次点击）

启动小程序 → 登录页 → [点击微信一键登录] → 填写昵称 → [点击确认] → 首页
→ [点击右上角"发布"] → 发布页 → 填写表单 → [点击发布] → 首页（帖子已显示）

核心流程 2：搭子加入并完成任务

首页 → [点击帖子卡片] → 帖子详情 → [点击加入搭子] → 确认提示
→ 任务进行中 → [搭子点击申请完成] → 发布者[标记完成] → 待评价状态
→ [双方各自提交证据] → [双方各自提交互评] → 已完成 → 积分结算提示

核心流程 3：发布者放弃任务

帖子详情 → [点击放弃任务] → 确认对话框（提示扣分金额）→ [确认放弃]
→ 帖子状态变为「已放弃」→ 积分扣除提示 → 返回首页

3.4.2 界面跳转关系

TabBar

- └─ 任务广场（首页）
 - └─ 帖子详情页 ↔ 发布页（编辑模式）
- └─ 排行榜页
- └─ 个人中心页
 - └─ 帖子详情页（历史帖子入口）

登录页 → 首页（登录成功后）

3.5 界面设计 UML 模型

3.5.1 界面导航图

```

[登录页]
  |
  | 登录成功
  ▼
[首页/任务广场] ←──────────────────┐
  | 点击帖子                          | 返回
  └──────────────────────────────────┘
  
```



3.5.2 边界类图

```

<<boundary>> LoginPage
+ wxLoginCode: string
+ nickname: string
+ avatarUrl: string
+ onClickLogin(): void
  
```

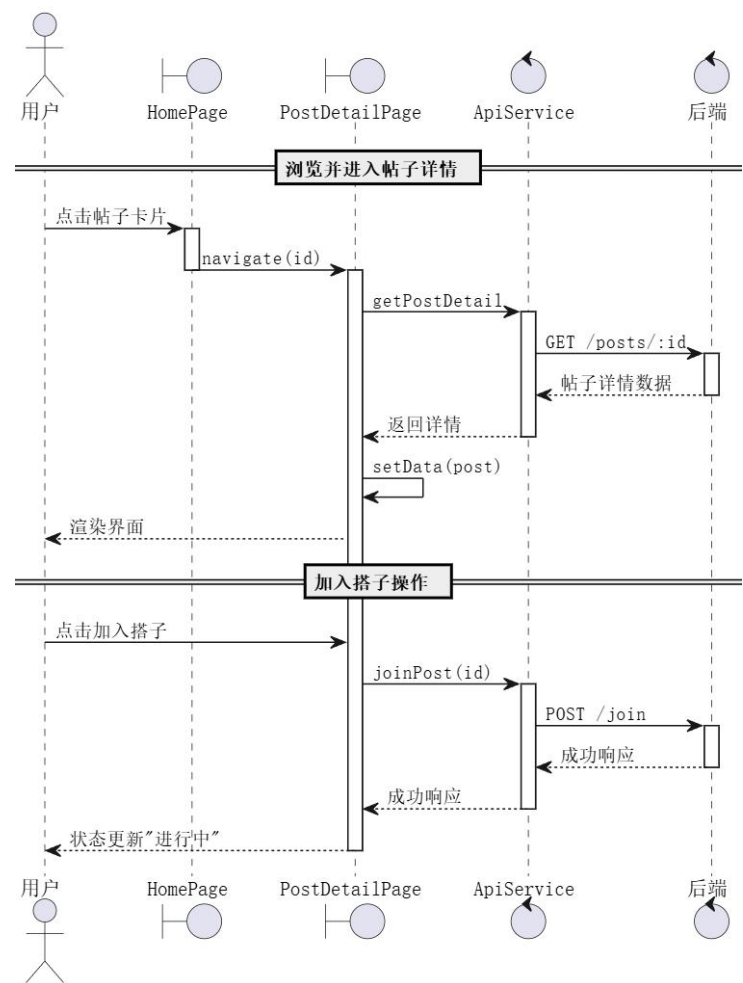
```

<<boundary>> HomePage
+ feed: Post[]
+ keyword: string
+ activeCategory: string
+ onSearch(): void
+ onLoadMore(): void
+ onClickPost(postId): void
  
```

```

<<boundary>> PostDetailPage
+ post: Post
+ buddies: Buddy[]
+ evidenceList: Evidence[]
+ onJoin(): void
+ onAbandon(): void
+ onSubmitEvidence(): void
+ onEvaluate(toId): void
  
```

3.5.3 关键界面交互顺序图（加入任务）



4 详细设计

4.1 用例详细设计

小组自选 3 个核心用例：发布任务贴（R03）、任务完成判定与积分结算（R10 - R15）、智能推荐排序（R21）。

4.1.1 用例 1：发布任务贴（R03）

执行步骤：

11. 用户在发布页填写表单（标题、内容、类别、积分、时间、最大搭子数）。
12. 前端校验：标题和内容不为空，积分 ≥ 0 ，结束时间 $>$ 开始时间。
13. 前端调用 `api.createPost(payload)`，`POST /api/posts`。
14. 后端校验发布者存在（查 `users` 表）。
15. 后端生成帖子 ID (`p_{timestamp}_{random6}`)，写入 `posts` 表，初始 `status = '招募中'`，`recommendedScore = 80`。

16. 返回完整帖子对象（含 id），前端刷新首页列表。

前置条件：用户已登录（currentUserId 非空）。

后置条件：posts 表新增一条记录，状态为「招募中」。

异常处理：

异常场景	处理方式
标题或内容为空	前端拦截，提示“标题和内容不能为空”
发布者 ID 不存在	后端返回 400，前端 toast 提示
数据库写入失败	后端返回 500，前端提示“发布失败，请重试”

4.1.2 用例 2：任务完成判定与积分结算（R10 - R15）

执行步骤：

17. 任务截止时间到达后，syncPostStatus 将状态从「进行中」变为「待评价」，并设置 evaluationDeadline = now + 32h。

18. 参与者（发布者和每位搭子）各自提交证据（POST /evidence），系统以（postId, submitterId）为唯一键，支持覆盖更新。

19. 参与者互相提交评价（POST /evaluate）：

- 校验：fromId ≠ toId，双方均为参与者，评分 1 - 5，不可重复评价同一人。
- 插入 evaluations 记录。
- 调用 updateUserAvgScore(toId) 更新被评价者均分。

20. 判定全员评价完成：检查是否所有（fromId, toId）配对均有记录（两人情况即发布者→搭子、搭子→发布者各一条）。

21. 最后一条评价提交后，事务内原子执行：

- UPDATE posts SET status='已完成', progress=100 WHERE id=? AND status='待评价'（乐观锁）
- UPDATE users SET points = points + reward WHERE id = publisherId
- 遍历 post_buddies，同样为每位搭子加积分

22. 调用 recalcCompletionRate(publisherId) 重算完成率。

前置条件：帖子状态为「待评价」，当前用户为参与者。

后置条件：帖子状态为「已完成」，发布者和搭子积分均增加，完成率重算。

异常处理：

异常场景	处理方式
重复评价同一人	后端返回 400，前端 toast 提示“已评价过”
评分不在 1 - 5	后端返回 400“评分须为 1-5 的整数”
乐观锁冲突（并发结算）	affectedRows === 0，跳过，防止重复加

	分
评价截止时间过期	canEvaluate = false, 前端隐藏评价按钮

4.1.3 用例 3: 智能推荐排序 (R21)

执行步骤:

- 前端调用 `api.getFeed({ userId, ... })`, GET `/api/posts?userId=xxx`。
- 后端查询全量候选帖子 (含发布者信息, LEFT JOIN users)。
- 若 `userId` 存在, 查询该用户历史参与帖子的类别分布 (`doneCount`、`abandonCount`) 构建 `preferenceMap`。
- 对每篇帖子调用 `calcRecommendedScore(post, publisherUser, preferenceMap)` 计算推荐分 (详见 4.4 算法设计)。
- 按推荐分降序内存排序, 截取当前页数据返回。
- 冷启动 (无历史数据) 时, `prefScore` 默认 50, 退化为基于帖子质量和发布者信誉的排序。

前置条件: 无 (游客可浏览, 不传 `userId` 则无个性化偏好)。

后置条件: 返回按推荐分排序的帖子分页列表, 每条附带动态 `recommendedScore`。

异常处理:

异常场景	处理方式
<code>userId</code> 不存在于数据库	<code>preferenceMap</code> 为 null, 退化为质量排序
数据库查询超时	Express 错误中间件返回 500

4.2 类设计

4.2.1 类识别与定义

系统共识别以下核心类 (对应后端业务实体):

类名	职责
User	用户信息、积分、完成率、AI 评价
Post	任务贴信息及状态机
PostBuddy	搭子关系记录
Evidence	任务完成证据
Evaluation	参与者互评记录
ApiService (前端)	封装所有 HTTP 请求, 返回 Promise
PostController (后端)	帖子相关路由处理, 协调业务逻辑
ScoreEngine (后端)	推荐分计算、积分结算、完成率重算
DbAdapter (后端)	连接池管理、query/withTransaction 封装

4.2.2 类关系说明

- User 1 —— * Post: 一个用户可发布多篇帖子 (publisherId 外键)。
- Post 1 —— * PostBuddy: 一篇帖子可有多多个搭子 (postId 外键)。
- User 1 —— * PostBuddy: 一个用户可加入多篇帖子 (userId 外键)。
- Post 1 —— * Evidence: 一篇帖子可有多条证据 (每参与者一条, 唯一约束)。
- Post 1 —— * Evaluation: 一篇帖子可有多条互评 (唯一约束 fromId+toId)。
- PostController 依赖 ScoreEngine (调用 settlePost、calcRecommendedScore)。
- PostController 依赖 DbAdapter (执行 SQL)。

4.2.3 关键类详细描述

Post 类 (对应 posts 表)

属性	类型	说明
id	String	主键, 格式 p_{ts}_{rand}
publisherId	String	FK → users.id
title	String	标题, ≤255 字符
content	String	正文
reward	Int	完成奖励积分
penalty	Int	放弃扣除积分
category	String	任务类别
status	String	状态机枚举
maxBuddies	Int	最大搭子数, 默认 1
currentBuddies	Int	当前搭子数
startTime	DateTime	任务开始时间
endTime	DateTime	任务截止时间
evaluationDeadline	DateTime	互评截止时间 (endTime + 32h)

方法:

方法	说明
syncStatus(now): void	检查时间条件, 按需流转状态
canJoin(userId): bool	校验是否可加入 (状态、人数、身份)
canAbandon(userId): bool	校验是否可放弃
canSubmitEvidence(userId, now): bool	校验是否可提交证据

ScoreEngine 类 (后端工具类)

方法	签名	说明
----	----	----

calcRecommendedScore	(post, publisherUser, preferenceMap): Int	三维加权推荐分
settlePost	(postId): Promise<void>	事务内积分结算
recalcCompletionRate	(userId): Promise<void>	重算完成率并写库
updateUserAvgScore	(userId): Promise<void>	重算均分并写库

4.3 数据设计

4.3.1 数据库概念结构设计 (E-R 图)



实体及主要属性:

- **User**: id(PK), openid(UNIQUE), nickname, avatarUrl, points, completionRate, avgScore, aiComment, createdAt
- **Post**: id(PK), publisherId(FK→User), title, content, reward, penalty, category, status, maxBuddies, currentBuddies, startTime, endTime, evaluationDeadline, createdAt
- **PostBuddy**: id(PK), postId(FK→Post), userId(FK→User), nickname, joinedAt; UNIQUE(postId, userId)
- **Evidence**: id(PK), postId(FK→Post), submitterId, submitterName, type, value, createdAt; UNIQUE(postId, submitterId)
- **Evaluation**: id(PK), postId(FK→Post), fromId(FK→User), fromName, toId(FK→User), score, content, createdAt; UNIQUE(postId, fromId, toId)

4.3.2 数据库逻辑结构设计 (表结构)

users 表

字段	类型	约束	说明
id	VARCHAR(64)	PK	u_{ts}_{rand}
openid	VARCHAR(100)	UNIQUE	微信唯一标识
nickname	VARCHAR(100)	NOT NULL	用户昵称
avatarUrl	TEXT	NOT NULL	头像 URL 或 Base64
points	INT	NOT NULL DEFAULT 0	当前积分 (≥0)
completionRate	INT	NOT NULL DEFAULT 0	完成率 0 - 100
avgScore	DECIMAL(3, 1)	NULL	收到的平均互评分

aiComment	TEXT	NOT NULL	AI 评价文字
createdAt	DATETIME	NOT NULL DEFAULT NOW()	注册时间

posts 表

字段	类型	约束	说明
id	VARCHAR(64)	PK	p_{ts}_{rand}
publisherId	VARCHAR(64)	NOT NULL, FK→users	发布者
publisherName	VARCHAR(100)	NOT NULL	发布者昵称（冗余）
title	VARCHAR(255)	NOT NULL	标题
content	TEXT	NOT NULL	正文
reward	INT	NOT NULL	完成奖励积分
penalty	INT	NOT NULL	放弃扣分
category	VARCHAR(50)	NOT NULL	类别
partnerChat	TINYINT(1)	NOT NULL DEFAULT 1	是否开启聊天
evaluationOpen	TINYINT(1)	NOT NULL DEFAULT 1	是否开启互评
evidenceText	TEXT	NOT NULL	证据要求说明
status	VARCHAR(50)	NOT NULL	状态枚举
buddyName	VARCHAR(100)	NOT NULL DEFAULT ,,	最新搭子昵称（冗余）
maxBuddies	INT	NOT NULL DEFAULT 1	最大搭子人数
currentBuddies	INT	NOT NULL DEFAULT 0	当前搭子人数
progress	INT	NOT NULL DEFAULT 0	任务进度 0 - 100
recommendedScore	INT	NOT NULL DEFAULT 80	初始推荐分（静态）
startTime	DATETIME	NULL	开始时间
endTime	DATETIME	NULL	截止时间
evaluationDeadline	DATETIME	NULL	互评截止时间
completionRequests	TEXT	NULL	搭子完成申请 JSON 数组
createdAt	DATETIME	NOT NULL DEFAULT NOW()	发布时间

post_buddies 表

字段	类型	约束	说明
id	VARCHAR(64)	PK	pb_{ts}_{rand}
postId	VARCHAR(64)	NOT NULL, FK→posts ON DELETE CASCADE	帖子 ID
userId	VARCHAR(64)	NOT NULL, FK→users ON	搭子用户 ID

		DELETE CASCADE	
nickname	VARCHAR(100)	NOT NULL	搭子昵称（冗余）
evaluated	TINYINT(1)	NOT NULL DEFAULT 0	是否已完成互评
joinedAt	DATETIME	NOT NULL DEFAULT NOW()	加入时间
UNIQUE	—	(postId, userId)	防止重复加入

evidences 表

字段	类型	约束	说明
id	VARCHAR(64)	PK	e_{ts}_{rand}
postId	VARCHAR(64)	NOT NULL, FK→posts	帖子 ID
submitterId	VARCHAR(64)	NOT NULL	提交者 ID
submitterName	VARCHAR(100)	NOT NULL	提交者昵称
type	VARCHAR(50)	NOT NULL	证据类型：文字 / 图片
value	TEXT	NOT NULL	证据内容
createdAt	DATETIME	NOT NULL DEFAULT NOW()	提交时间
UNIQUE	—	(postId, submitterId)	每人每帖仅一条（可覆盖）

evaluations 表

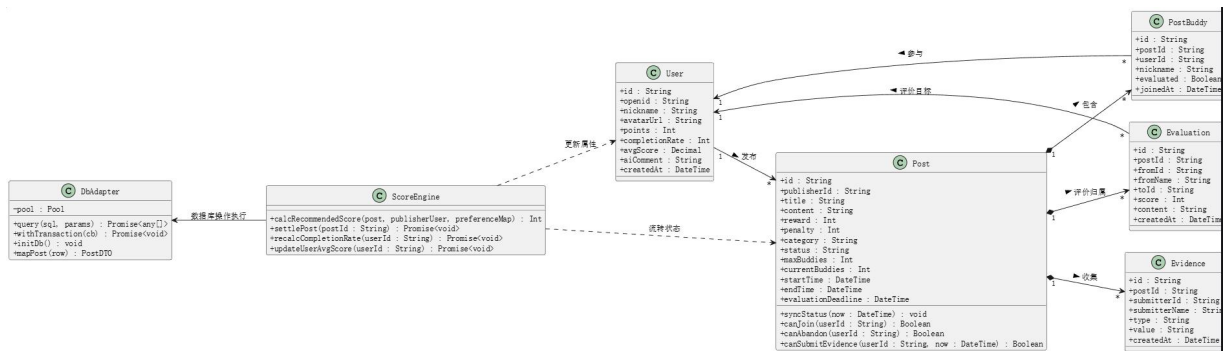
字段	类型	约束	说明
id	VARCHAR(64)	PK	ev_{ts}_{rand}
postId	VARCHAR(64)	NOT NULL, FK→posts	帖子 ID
fromId	VARCHAR(64)	NULL, FK→users	评价人 ID
fromName	VARCHAR(100)	NOT NULL	评价人昵称
toId	VARCHAR(64)	NOT NULL	被评价者 ID
score	INT	NOT NULL	评分 1 - 5
content	TEXT	NOT NULL	评语
createdAt	DATETIME	NOT NULL DEFAULT NOW()	评价时间
UNIQUE	—	(postId, fromId, toId)	防止重复评价
INDEX	—	idx_eval_toId (toId)	加速 WHERE toId=? 查询

4.3.3 数据存储与访问策略

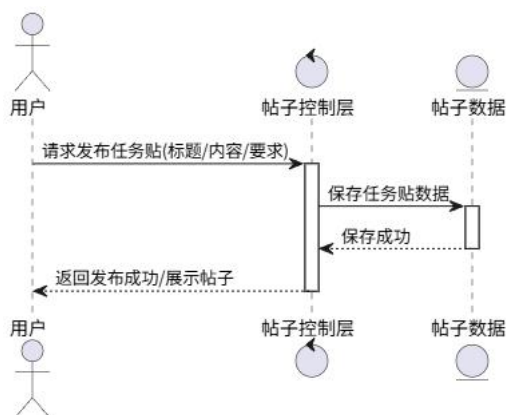
- 连接池: connectionLimit: 10, queueLimit: 0 (无限排队等待), 字符集 utf8mb4, 支持中文及 emoji。
- 预处理语句: 所有 SQL 通过 connection.execute(sql, params) 执行, 防止 SQL 注入。
- 外键级联: posts 删除时级联删除 post_buddies、evidences、evaluations (ON DELETE CASCADE)。
- 索引策略:
 - users.openid UNIQUE 索引 (登录查询)
 - post_buddies.(postId, userId) UNIQUE (加入校验)
 - evidences.(postId, submitterId) UNIQUE (证据唯一性)
 - evaluations.(postId, fromId, toId) UNIQUE (防重复评价)
 - evaluations.toId 普通索引 (收到的评价列表查询)
- 兼容性迁移: initDb 启动时检查列是否存在 (SHOW COLUMNS FROM ... LIKE '...'), 缺失则 ALTER TABLE 补加, 实现零停机迁移。

4.4 详细设计 UML 模型

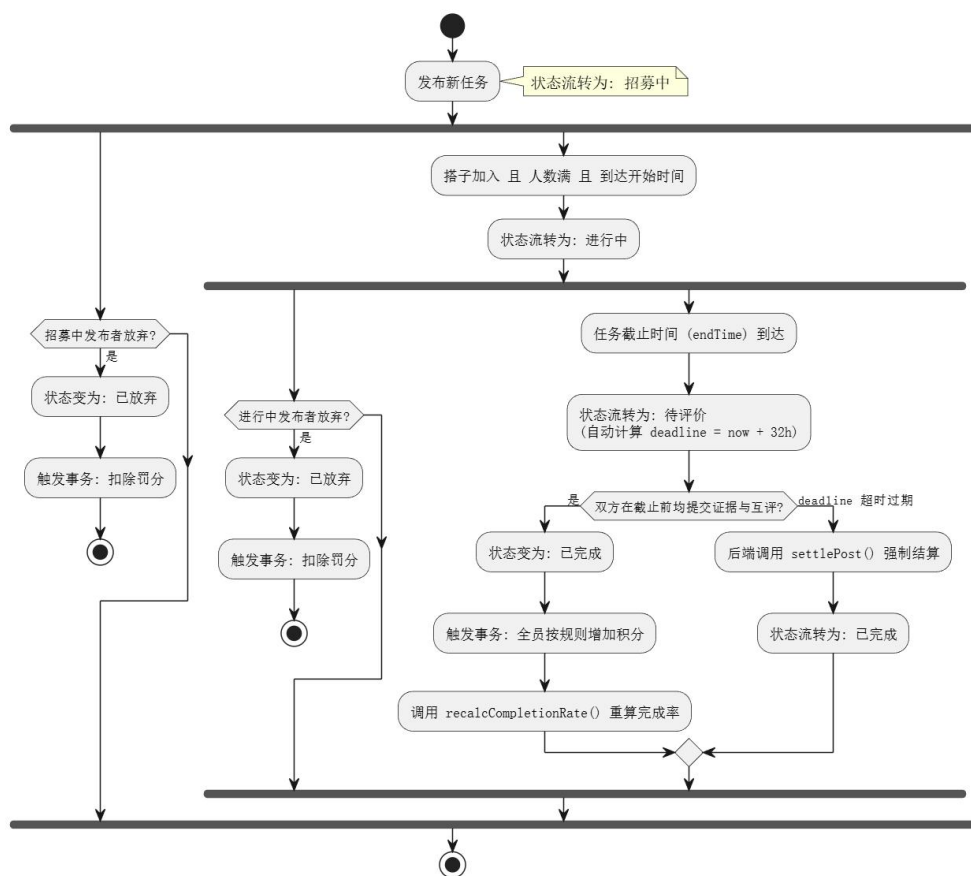
4.4.1 详细类图



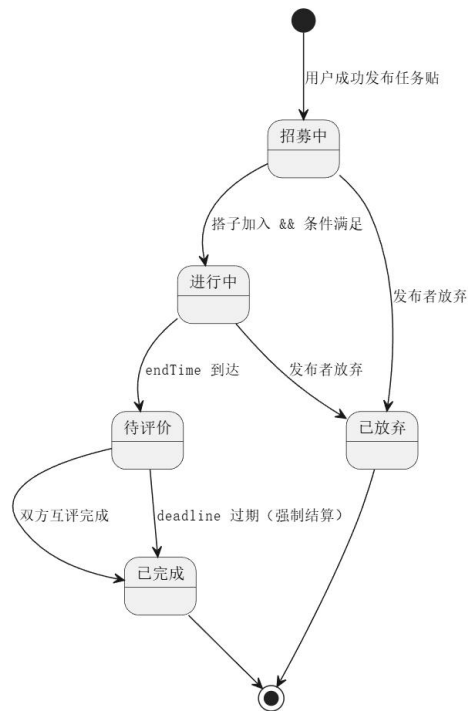
4.4.2 用例详细顺序图（发布任务贴）



4.4.3 关键用例活动图（任务完整生命周期）



4.4.4 状态图（帖子状态机）



4.4.3 关键算法：推荐分计算（`calcRecommendedScore`）

输入： 帖子对象 `post`，发布者信息 `publisherUser`，用户偏好向量 `preferenceMap`

输出： 0 - 100 的整数推荐分

伪代码：

```
function calcRecommendedScore(post, publisherUser, preferenceMap):
    // 发布者信用分 (0-100)
    cr = publisherUser ? publisherUser.completionRate : 0           // 0-100
    pts = publisherUser ? min(publisherUser.points / 10, 100) : 0
    publisherScore = cr * 0.6 + pts * 0.4                           // 0-100

    // 帖子质量分
    rewardScore = min(post.reward / 2, 50)                         // 0-50
    penaltyScore = min(post.penalty / 2, 30)                       // 0-30
    hotBonus = (post.currentBuddies >= post.maxBuddies * 0.8) ? 20 : 0
    postScore = rewardScore + penaltyScore + hotBonus               // 0-100

    // 用户偏好分 (0-100)
    pref = preferenceMap ? preferenceMap.get(post.category) : null
    if pref != null and (pref.doneCount * 2 + pref.abandonCount) > 0:
        prefScore = (pref.doneCount * 2) / (pref.doneCount * 2 +
        pref.abandonCount) * 100
    else:
        prefScore = 50      // 冷启动默认
```

```
return round(publisherScore * 0.4 + postScore * 0.3 + prefScore * 0.3)
```

设计说明:

- 完成率权重 (0.6) 高于积分权重 (0.4)，因为完成率是更稳定的信用指标。
- 偏好分使用成功率公式 (完成计 2 分、放弃计 1 分)，给完成任务的用户更高权重。
- 热度加成 (80% 满员时 +20 分) 鼓励快速组队。
- 三维权重 ($0.4 + 0.3 + 0.3 = 1.0$)，产出 0 - 100 区间。

5 设计总结

5.1 设计成果概述

本次设计完整覆盖了 Task Buddy 三个设计阶段:

29. **系统架构设计**: 确立微信小程序 + Express + MySQL 前后端分离三层架构，完成 6 个前端模块和 8 个后端模块的划分，定义全部 REST API 接口契约。
30. **界面设计**: 制定校园清新风格规范，设计 6 个核心页面的原型布局，梳理 3 条核心交互流程 (均在 3 次点击内完成)。
31. **详细设计**: 细化 3 个核心用例的完整执行流程，完成 5 张数据库表的逻辑结构设计，设计推荐分算法和积分结算事务逻辑。

5.2 设计优势与不足

优势:

- 帖子状态机设计完整，5 种状态的转换条件和副作用均有明确约束。
- 积分结算采用乐观锁机制，在单机场景下有效防止并发重复结算。
- 推荐算法三维加权，兼顾冷启动 (无偏好数据) 和个性化两种场景。
- 数据库设计遵循范式，冗余字段 (publisherName、buddyName) 有明确的业务理由 (减少 JOIN 次数)。

不足:

- 推荐排序采用全量内存排序，当帖子量大时性能下降，需改为数据库索引排序或预计算。
- 帖子状态同步依赖请求触发 (syncPostStatus)，状态变更不实时，高严格性场景需引入定时任务。
- 聊天功能 (R09) 在当前架构中仅有需求描述，未实现 WebSocket 或轮询方案，是后续版本的主要欠缺。
- 微信内容安全接口的集成在本文档中已规划，但在当前实现中尚未在帖子发布流程中调用。

5.3 后续工作展望

- 32. 接入微信内容安全 API，在 POST /api/posts 中增加文本审核步骤。
- 33. 实现搭子聊天功能（R09），推荐使用 WebSocket 或微信订阅消息方案。
- 34. 优化推荐算法性能：将 recommendedScore 异步预计算并库存，读时直接 ORDER BY recommendedScore DESC。
- 35. 引入定时任务（Node.js node-cron）定期调用 syncPostStatus，实现帖子状态自动流转。
- 36. 完善 AI 评价模块（R20）：基于积分变动、完成率、均分生成结构化文字评价。

参考文献

37. 《校园搭子任务互助平台——需求规格说明书》v1.0，第 5 组，2026-05-13

38. Express.js 官方文档，<https://expressjs.com/>

39. 微信小程序官方开发文档，
<https://developers.weixin.qq.com/miniprogram/dev/framework/>

40. MySQL 8.0 Reference Manual，<https://dev.mysql.com/doc/refman/8.0/en/>

41. mysql2 驱动文档，<https://github.com/sidorares/node-mysql2>

42. UML 2.0 规范，Object Management Group

附录

附录 A 系统架构设计评审表

评审项	评审结论	评审意见
架构选型理由是否充分	☐ 通过 ☐ 不通过	
模块划分是否清晰	☐ 通过 ☐ 不通过	
接口定义是否规范	☐ 通过 ☐ 不通过	
设计模型是否完整准确	☐ 通过 ☐ 不通过	
评审结论	☐ 通过 ☐ 修改后通过 ☐ 不通过	
评审人签名		
评审日期		

附录 B 界面设计评审表

评审项	评审结论	评审意见
界面风格是否统一美观	☐ 通过 ☐ 不通过	
交互流程是否合理用户友好	☐ 通过 ☐ 不通过	
原型图是否完整清晰	☐ 通过 ☐ 不通过	
设计模型是否符合要求	☐ 通过 ☐ 不通过	
评审结论	☐ 通过 ☐ 修改后通过 ☐ 不	

	通过	
评审人签名		
评审日期		

附录 C 详细设计评审表

评审项	评审结论	评审意见
用例设计是否全面	☐ 通过 ☐ 不通过	
类图是否完整准确	☐ 通过 ☐ 不通过	
算法逻辑是否清晰	☐ 通过 ☐ 不通过	
数据设计是否合理	☐ 通过 ☐ 不通过	
各种 UML 图绘制是否规范	☐ 通过 ☐ 不通过	
评审结论	☐ 通过 ☐ 修改后通过 ☐ 不通过	
评审人签名		
评审日期		

文档版本：v1.0 | 编写日期：2026-05-24 | 第 5 组